CSA4704 – DEEP LEARNING

Name: PRAHARSHINI Register Number: 192424085

EXPERIMENT 11: CNN FOR IMAGE CLASSIFICATION

Aim: To build a Convolutional Neural Network that classifies images into ten categories using the CIFAR-10 dataset.

Algorithm

1. Load CIFAR-10 and normalise pixel values.
2. Split training data into training and validation sets.
3. Build a CNN using convolution, pooling, flatten and dense layers.
4. Train using categorical cross-entropy.
5. Display accuracy/loss, confusion matrix, wrong predictions and first-layer feature maps.

Python Code

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay


(x, y), (xt, yt) = tf.keras.datasets.cifar10.load_data()
names = ["airplane","car","bird","cat","deer","dog","frog","horse","ship","truck"]
x, xt = x.astype("float32")/255, xt.astype("float32")/255
y, yt = y.ravel(), yt.ravel()
xtr, xval, ytr, yval = x[:45000], x[45000:], y[:45000], y[45000:]

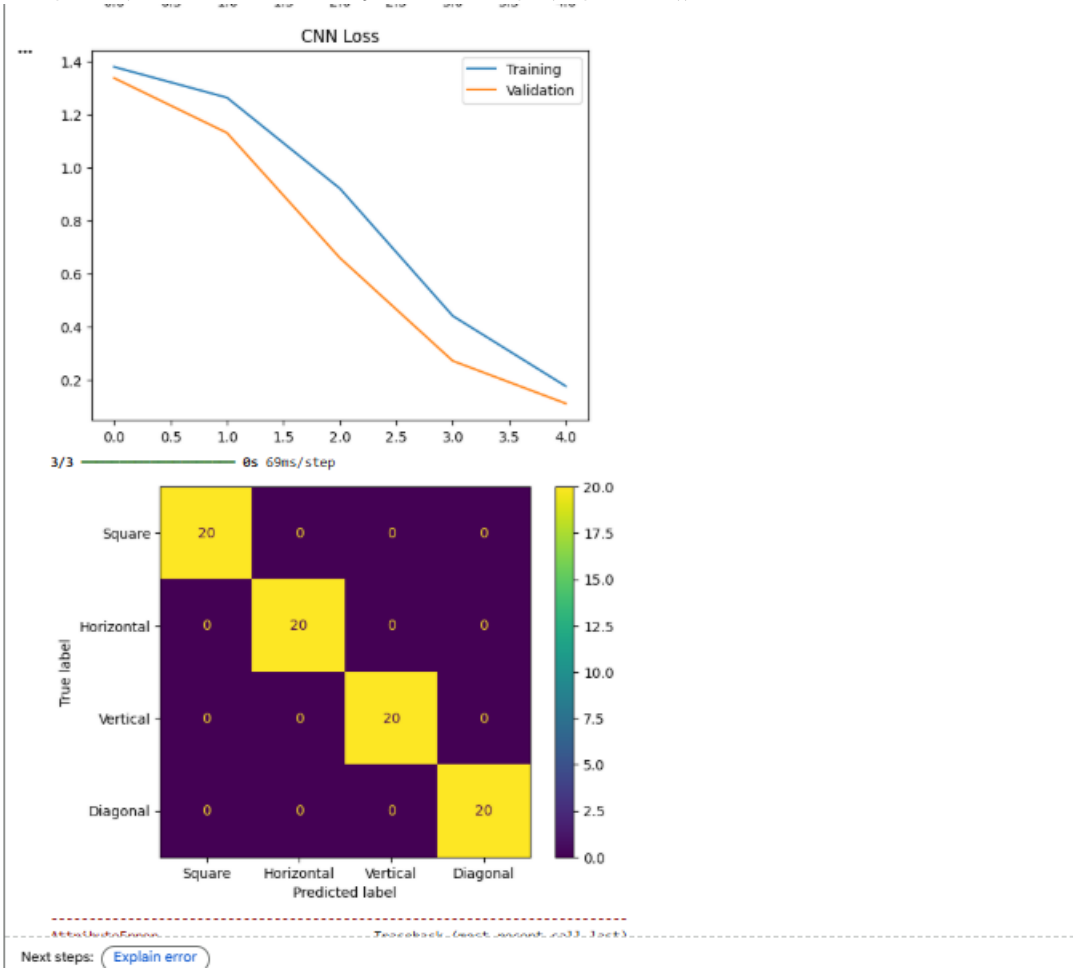

model = tf.keras.Sequential([
    tf.keras.layers.Input((32,32,3)),
    tf.keras.layers.Conv2D(32, 3, activation="relu", name="conv1"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(64, 3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
h = model.fit(xtr, ytr, validation_data=(xval,yval),
              epochs=5, batch_size=64)


plt.plot(h.history["accuracy"], label="Train")
plt.plot(h.history["val_accuracy"], label="Validation")
plt.legend(); plt.title("Accuracy"); plt.show()
plt.plot(h.history["loss"], label="Train")
plt.plot(h.history["val_loss"], label="Validation")
plt.legend(); plt.title("Loss"); plt.show()


pred = model.predict(xt, verbose=0).argmax(axis=1)
cm = confusion_matrix(yt, pred)
ConfusionMatrixDisplay(cm, display_labels=names).plot(xticks_rotation=45)
plt.show()
```

```
wrong = np.where(pred != yt)[0][:5]
for i in wrong:
    plt.imshow(xt[i]); plt.title(f"Actual: {names[yt[i]]}, Predicted: {names[pred[i]]}")
    plt.axis("off"); plt.show()


feature_model = tf.keras.Model(model.input, model.get_layer("conv1").output)
fm = feature_model.predict(xt[:1], verbose=0)[0]
fig, axes = plt.subplots(1, 6, figsize=(12,2))
for i, ax in enumerate(axes):
    ax.imshow(fm[:,:,i], cmap="viridis"); ax.axis("off")
plt.suptitle("First Convolution Layer Feature Maps"); plt.show()
```



Result

Result: The CNN was trained to classify CIFAR-10 images. Accuracy and loss curves, a confusion matrix, sample incorrect predictions and first-layer feature maps were displayed.

EXPERIMENT 12: CNN ARCHITECTURE AND HYPERPARAMETER COMPARISON

Aim: To compare CNN architectures and training settings using accuracy, parameter count and training time.

Algorithm

1. Load and normalise CIFAR-10 images.
2. Create CNN models with 2, 3 or 4 convolution layers.
3. Change filter size, filter count, pooling and batch normalisation.
4. Train each model using selected learning rate and batch size.
5. Record test accuracy, parameter count and training time in a table.

Python Code

```
import tensorflow as tf
import pandas as pd
import time


(x,y),(xt,yt) = tf.keras.datasets.cifar10.load_data()
x,xt = x[:12000]/255.0,xt[:2000]/255.0
y,yt = y[:12000].ravel(),yt[:2000].ravel()


def build_model(layers=2, k=3, filters=32, pool="max", bn=False):
    m = tf.keras.Sequential([tf.keras.layers.Input((32,32,3))])
    for i in range(layers):
        m.add(tf.keras.layers.Conv2D(filters*(i+1), k, padding="same", activation="relu"))
        if bn: m.add(tf.keras.layers.BatchNormalization())
        m.add(tf.keras.layers.MaxPooling2D() if pool=="max"
              else tf.keras.layers.AveragePooling2D())
    m.add(tf.keras.layers.Flatten())
    m.add(tf.keras.layers.Dense(64, activation="relu"))
    m.add(tf.keras.layers.Dense(10, activation="softmax"))
    return m


rows = []
configs = [(2,3,32,"max",False,0.001,64),
           (3,3,32,"max",True,0.001,64),
           (4,3,32,"avg",True,0.001,64),
           (2,5,48,"avg",False,0.0005,128)]
for n,k,f,p,bn,lr,bs in configs:
    m = build_model(n,k,f,p,bn)
    m.compile(optimizer=tf.keras.optimizers.Adam(lr),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    start = time.time()
    m.fit(x,y,epochs=2,batch_size=bs,verbose=0)
    loss,acc = m.evaluate(xt,yt,verbose=0)
    rows.append([n,f"{k}x{k}",f,p,bn,lr,bs,round(acc,4),
                 m.count_params(),round(time.time()-start,2)])
result = pd.DataFrame(rows, columns=["Conv layers","Filter size","Filters",
    "Pooling","BatchNorm","Learning rate","Batch size","Test accuracy",
    "Parameters","Training seconds"])
print(result.to_string(index=False))
```

```
import tensorflow as tf
import pandas as pd
import time

(x,y),(xt,yt) = tf.keras.datasets.cifar10.load_data()
x,xt = x[:12000]/255.0,xt[:2000]/255.0
y,yt = y[:12000].ravel(),yt[:2000].ravel()

def build_model(layers=2, k=3, filters=32, pool="max", bn=False):
    m = tf.keras.Sequential([tf.keras.layers.Input((32,32,3))])
    for i in range(layers):
        m.add(tf.keras.layers.Conv2D(filters*(i+1), k, padding="same", activation="relu"))
        if bn: m.add(tf.keras.layers.BatchNormalization())
        m.add(tf.keras.layers.MaxPooling2D(pool_size=(2, 2)) if pool=="max"
              else tf.keras.layers.AveragePooling2D(pool_size=(2, 2)))
    m.add(tf.keras.layers.Flatten())
    m.add(tf.keras.layers.Dense(64, activation="relu"))
    m.add(tf.keras.layers.Dense(10, activation="softmax"))
    return m

rows = []
configs = [(2,3,32,"max",False,0.001,64),
           (3,3,32,"max",True,0.001,64),
           (4,3,32,"avg",True,0.001,64),
           (2,5,48,"avg",False,0.0005,128)]
for n,k,f,p,bn,lr,bs in configs:
    m = build_model(n,k,f,p,bn)
    m.compile(optimizer=tf.keras.optimizers.Adam(lr),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    start = time.time()
    m.fit(x,y,epochs=2,batch_size=bs,verbose=0)
    loss,acc = m.evaluate(xt,yt,verbose=0)
    rows.append([n,f"{k}x{k}",f,p,bn,lr,bs,round(acc,4),
                 m.count_params(),round(time.time()-start,2)])
result = pd.DataFrame(rows, columns=["Conv layers","Filter size","Filters",
    "Pooling","BatchNorm","Learning rate","Batch size","Test accuracy",
    "Parameters","Training seconds"])
print(result.to_string(index=False))
```

```
 Conv layers Filter size  Filters Pooling  BatchNorm  Learning rate  Batch size  Test accuracy  Parameters  Training seconds
           2         3x3       32     max      False         0.0010          64         0.4795      282250             45.83
           3         3x3       32     max       True         0.0010          64         0.2295      174570             87.35
           4         3x3       32     avg       True         0.0010          64         0.1130      220266             87.01
           2         5x5       48     avg      False         0.0005         128         0.4405      512874            127.51
```

Result

Result: The configurations were compared using test accuracy, parameter count and training time. The table helps select a suitable model for a low-cost device.

EXPERIMENT 13: SEQUENTIAL DATASET PREPARATION AND RNN PREDICTION

Aim: To prepare time-series sequences and predict the next-hour electricity demand using a SimpleRNN.

Algorithm

1. Create or load hourly demand data with Pandas.
2. Fill missing values and scale the series.
3. Create windows of 24 past hours to predict the next hour.
4. Split sequences chronologically.
5. Train SimpleRNN and calculate MAE and RMSE.

Python Code

```
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

```
np.random.seed(4)
t = np.arange(24*120)
series = 500 + 80*np.sin(2*np.pi*t/24) + 20*np.sin(2*np.pi*t/(24*7)) + np.random.randn(len(t))*8
df = pd.DataFrame({"demand":series}).interpolate()
scaler = MinMaxScaler()
z = scaler.fit_transform(df[["demand"]]).ravel()


X,y = [],[]
for i in range(24,len(z)):
    X.append(z[i-24:i]); y.append(z[i])
X,y = np.array(X)[...,None],np.array(y)
cut = int(len(X)*0.8)
Xtr,Xte,ytr,yte = X[:cut],X[cut:],y[:cut],y[cut:]


model = tf.keras.Sequential([
    tf.keras.layers.Input((24,1)),
    tf.keras.layers.SimpleRNN(32),
    tf.keras.layers.Dense(1)
])
model.compile(optimizer="adam",loss="mse")
model.fit(Xtr,ytr,epochs=8,batch_size=32,verbose=0)
pred = model.predict(Xte,verbose=0).ravel()
actual = scaler.inverse_transform(yte.reshape(-1,1)).ravel()
forecast = scaler.inverse_transform(pred.reshape(-1,1)).ravel()
print("MAE:",mean_absolute_error(actual,forecast))
print("RMSE:",np.sqrt(mean_squared_error(actual,forecast)))
plt.plot(actual,label="Actual"); plt.plot(forecast,label="Predicted")
plt.legend(); plt.title("Hourly Demand Forecast"); plt.show()
```

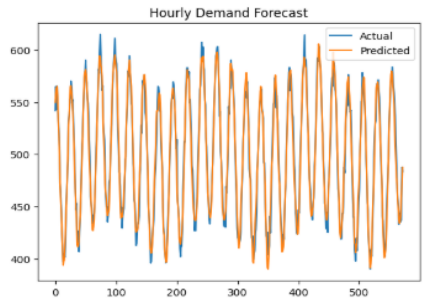
```
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

np.random.seed(4)
t = np.arange(24*120)
series = 500 + 80*np.sin(2*np.pi*t/24) + 20*np.sin(2*np.pi*t/(24*7)) + np.random.randn(len(t))*8
df = pd.DataFrame({"demand":series}).interpolate()
scaler = MinMaxScaler()
z = scaler.fit_transform(df[["demand"]]).ravel()

X,y = [],[]
for i in range(24,len(z)):
    X.append(z[i-24:i]); y.append(z[i])
X,y = np.array(X)[...,None],np.array(y)
cut = int(len(X)*0.8)
Xtr,Xte,ytr,yte = X[:cut],X[cut:],y[:cut],y[cut:]

model = tf.keras.Sequential([
    tf.keras.layers.Input((24,1)),
    tf.keras.layers.SimpleRNN(32),
    tf.keras.layers.Dense(1)
])
model.compile(optimizer="adam",loss="mse")
model.fit(Xtr,ytr,epochs=8,batch_size=32,verbose=0)
pred = model.predict(Xte,verbose=0).ravel()
actual = scaler.inverse_transform(yte.reshape(-1,1)).ravel()
forecast = scaler.inverse_transform(pred.reshape(-1,1)).ravel()
print("MAE:",mean_absolute_error(actual,forecast))
print("RMSE:",np.sqrt(mean_squared_error(actual,forecast)))
plt.plot(actual,label="Actual"); plt.plot(forecast,label="Predicted")
plt.legend(); plt.title("Hourly Demand Forecast"); plt.show()
```

```
MAE: 8.165223733006451
RMSE: 10.332882832593086
```



Result

Result: The SimpleRNN predicted the next-hour demand from the previous 24 hours. Actual and predicted values were plotted and MAE/RMSE were calculated.

EXPERIMENT 14: UNFOLDED RNN ARCHITECTURE VISUALIZATION

Aim: To visualise a SimpleRNN unrolled over five time steps and verify manually calculated hidden states against Keras.

Algorithm

1. Create a five-step input sequence and a SimpleRNN with three hidden units.
2. Extract input, recurrent and output weights.
3. Calculate each hidden state using the RNN equation.
4. Compare manual states with Keras output.
5. Draw the unfolded network and plot hidden-state values.

Python Code

```
import tensorflow as tf
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt


tf.random.set_seed(2)
seq = np.array([[[1.],[2.],[3.],[4.],[5.]]],dtype="float32")
rnn = tf.keras.layers.SimpleRNN(3, return_sequences=True, return_state=True)
dense = tf.keras.layers.Dense(1)
out, state = rnn(seq)
pred = dense(out)
Wx, Wh, bias = rnn.get_weights()
Wy, by = dense.get_weights()


h = np.zeros(3)
manual = []
for x in seq[0]:
    h = np.tanh(x @ Wx + h @ Wh + bias)
    manual.append(h.copy())
manual = np.array(manual)
print("Manual hidden states:\n",np.round(manual,4))
print("Keras hidden states:\n",np.round(out.numpy()[0],4))
print("Maximum difference:",np.max(np.abs(manual-out.numpy()[0])))


G = nx.DiGraph()
for t in range(1,6):
    G.add_node(f"x{t}",layer=0); G.add_node(f"h{t}",layer=1); G.add_node(f"y{t}",layer=2)
    G.add_edge(f"x{t}",f"h{t}",label="W_xh")
    G.add_edge(f"h{t}",f"y{t}",label="W_hy")
    if t>1: G.add_edge(f"h{t-1}",f"h{t}",label="W_hh")
pos = {}
for t in range(1,6):
    pos[f"x{t}"]=(t,2); pos[f"h{t}"]=(t,1); pos[f"y{t}"]=(t,0)
nx.draw(G,pos,with_labels=True,node_size=900,node_color="lightblue",arrows=True)
nx.draw_networkx_edge_labels(G,pos,edge_labels=nx.get_edge_attributes(G,"label"),font_size=7)
plt.title("SimpleRNN Unfolded Across Five Time Steps"); plt.show()
plt.plot(manual,marker="o")
plt.xlabel("Time step"); plt.ylabel("Hidden-state value")
plt.title("Hidden State Values"); plt.grid(); plt.show()
```

```
...          Method  Train Accuracy  Test Accuracy  Train-Test Gap
    0      Baseline          0.9425           0.90          0.0425
    1            L2          0.9425           0.91          0.0325
    2       Dropout          0.9275           0.90          0.0275
    3  EarlyStopping          0.9400           0.91          0.0300
```

Result

Result: The RNN was unfolded into five time steps. Manual hidden states were compared with Keras states, and the network graph and hidden-state plot were displayed.

EXPERIMENT 15: BPTT FLOW AND GRADIENT ANALYSIS

Aim: To study vanishing and exploding gradients in a SimpleRNN for different sequence lengths and activation functions.

Algorithm

1. Generate a time series and create sequences of 10, 50 and 100 steps.
2. Build SimpleRNN models using tanh and ReLU.
3. Use GradientTape to calculate loss gradients.
4. Record gradient norms at recurrent time steps.
5. Plot gradient magnitude and compare the two activations.

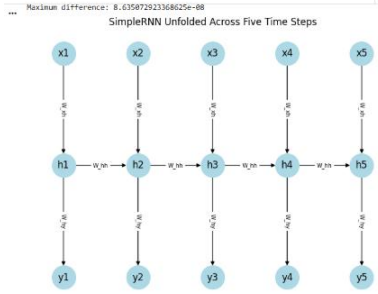
Python Code

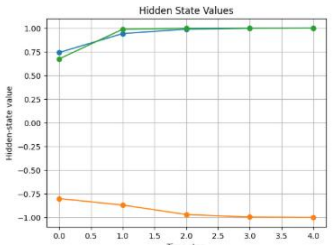
```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt


np.random.seed(3)
for activation in ["tanh","relu"]:
    for length in [10,50,100]:
        model = tf.keras.Sequential([
            tf.keras.layers.Input((length,1)),
            tf.keras.layers.SimpleRNN(16,activation=activation),
            tf.keras.layers.Dense(1)
        ])
        x = tf.Variable(np.random.randn(8,length,1).astype("float32"))
        target = tf.zeros((8,1))
        with tf.GradientTape() as tape:
            y = model(x,training=True)
            loss = tf.reduce_mean((y-target)**2)
        grad = tape.gradient(loss,x)
        # Norm by input time step is used as a practical gradient trace.
        norms = tf.norm(grad,axis=[0,2]).numpy()
        plt.plot(norms,label=f"{activation}, length={length}")
plt.yscale("log")
plt.xlabel("Time step"); plt.ylabel("Gradient norm (log scale)")
plt.title("Gradient Magnitude Across Sequence Positions")
plt.legend(); plt.show()
```

```
    G.add_node(f"x{t}",layer=0); G.add_node(f"h{t}",layer=1); G.add_node(f"y{t}",layer=2)
    G.add_edge(f"x{t}",f"h{t}",label="W_xh")
    G.add_edge(f"h{t}",f"y{t}",label="W_hy")
    if t>1: G.add_edge(f"h{t-1}",f"h{t}",label="W_hh")
pos = {}
for t in range(1,6):
    pos[f"x{t}"]=(t,2); pos[f"h{t}"]=(t,1); pos[f"y{t}"]=(t,0)
nx.draw(G,pos,with_labels=True,node_size=900,node_color="lightblue",arrows=True)
nx.draw_networkx_edge_labels(G,pos,edge_labels=nx.get_edge_attributes(G,"label"),font_size=7)
plt.title("SimpleRNN Unfolded Across Five Time Steps"); plt.show()
plt.plot(manual,marker="o")
plt.xlabel("Time step"); plt.ylabel("Hidden-state value")
plt.title("Hidden State Values"); plt.grid(); plt.show()
```

```
Maximum difference: 8.635072923368625e-08
```





Result

Result: Gradient norms were measured for sequence lengths 10, 50 and 100 using tanh and ReLU. Very small values indicate vanishing gradients, while unusually large values indicate possible exploding gradients.

EXPERIMENT 16: LSTM AND RNN PERFORMANCE COMPARISON

Aim: To compare SimpleRNN and LSTM for forecasting daily order volume using a 30-day look-back window.

Algorithm

1. Generate a daily series with weekly and seasonal patterns.
2. Scale the values and create 30-day input windows.
3. Train SimpleRNN and LSTM with the same number of units.
4. Measure training time and test MAE, RMSE and MAPE.
5. Plot forecasts and display a Pandas comparison table.

Python Code

```
import numpy as np, pandas as pd, tensorflow as tf, time
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

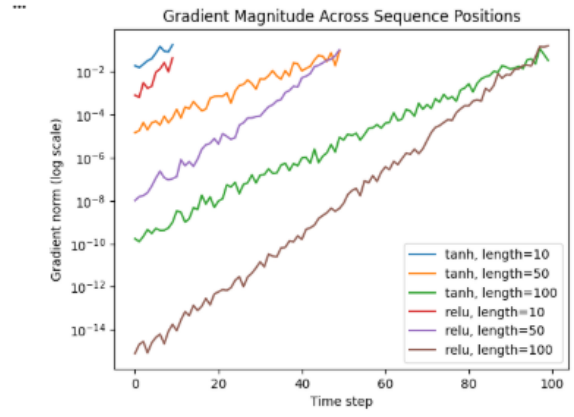

np.random.seed(5)
t=np.arange(730)
values=300+40*np.sin(2*np.pi*t/7)+70*np.sin(2*np.pi*t/365)+np.random.randn(730)*10
sc=MinMaxScaler(); z=sc.fit_transform(values.reshape(-1,1)).ravel()
X,y=[],[]
for i in range(30,len(z)): X.append(z[i-30:i]); y.append(z[i])
```

```
X=np.array(X)[...,None]; y=np.array(y)
cut=int(.8*len(X)); Xtr,Xte=X[:cut],X[cut:]; ytr,yte=y[:cut],y[cut:]


rows=[]; forecasts={}
for kind in ["SimpleRNN","LSTM"]:
    layer=tf.keras.layers.SimpleRNN(32) if kind=="SimpleRNN" else tf.keras.layers.LSTM(32)
    m=tf.keras.Sequential([tf.keras.layers.Input((30,1)),layer,tf.keras.layers.Dense(1)])
    m.compile(optimizer="adam",loss="mse")
    start=time.time()
    hist=m.fit(Xtr,ytr,validation_split=.2,epochs=8,batch_size=32,verbose=0)
    seconds=time.time()-start
    p=m.predict(Xte,verbose=0).ravel()
    a=sc.inverse_transform(yte.reshape(-1,1)).ravel()
    p=sc.inverse_transform(p.reshape(-1,1)).ravel()
    mae=mean_absolute_error(a,p); rmse=np.sqrt(mean_squared_error(a,p))
    mape=np.mean(np.abs((a-p)/a))*100
    rows.append([kind,mae,rmse,mape,seconds])
    forecasts[kind]=p
    plt.plot(hist.history["loss"],label=kind+" train")
    plt.plot(hist.history["val_loss"],"--",label=kind+" validation")
plt.legend(); plt.title("Training and Validation Loss"); plt.show()
actual=sc.inverse_transform(yte.reshape(-1,1)).ravel()
plt.plot(actual,label="Actual")
for k,p in forecasts.items(): plt.plot(p,label=k)
plt.legend(); plt.title("Order Volume Forecast"); plt.show()
print(pd.DataFrame(rows,columns=["Model","MAE","RMSE","MAPE (%)","Training time (s)"]))
```

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(3)
for activation in ["tanh","relu"]:
    for length in [10,50,100]:
        model = tf.keras.Sequential([
            tf.keras.layers.Input((length,1)),
            tf.keras.layers.SimpleRNN(16,activation=activation),
            tf.keras.layers.Dense(1)
        ])
        x = tf.Variable(np.random.randn(8,length,1).astype("float32"))
        target = tf.zeros((8,1))
        with tf.GradientTape() as tape:
            y = model(x,training=True)
            loss = tf.reduce_mean((y-target)**2)
        grad = tape.gradient(loss,x)
        # Norm by input time step is used as a practical gradient trace.
        norms = tf.norm(grad,axis=[0,2]).numpy()
        plt.plot(norms,label=f"{activation}, length={length}")
plt.yscale("log")
plt.xlabel("Time step"); plt.ylabel("Gradient norm (log scale)")
plt.title("Gradient Magnitude Across Sequence Positions")
plt.legend(); plt.show()
```



Result

Result: SimpleRNN and LSTM were trained on the same sequence data. Their loss curves, forecasts, MAE, RMSE, MAPE and training times were compared.

EXPERIMENT 17: RNN GRADIENT ANALYSIS AND GRADIENT CLIPPING

Aim: To detect large gradients during character-level text training and stabilise the RNN using clipnorm and clipvalue.

Algorithm

1. Prepare a small character corpus and convert characters to integer IDs.
2. Train a SimpleRNN with a deliberately high learning rate without clipping.
3. Record global gradient norm and loss.
4. Repeat with clipnorm and clipvalue.
5. Plot the results and select a stable clipping setting.

Python Code

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

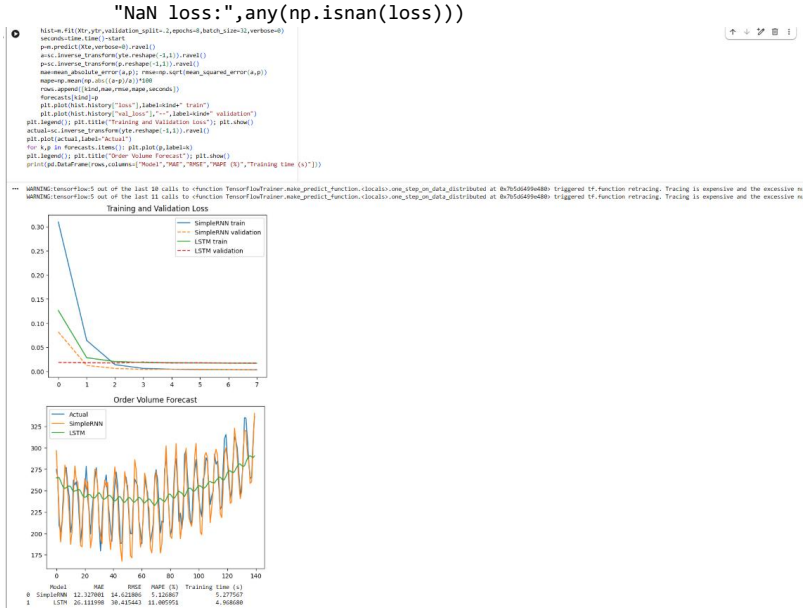

text = ("deep learning models learn patterns from data. " * 200)
chars=sorted(set(text)); to_id={c:i for i,c in enumerate(chars)}
ids=np.array([to_id[c] for c in text])
seq_len=20
X=np.array([ids[i:i+seq_len] for i in range(len(ids)-seq_len)])
Y=np.array([ids[i+seq_len] for i in range(len(ids)-seq_len)])
X,Y=X[:3000],Y[:3000]
vocab=len(chars)


def train(clip=None, mode=None):
    model=tf.keras.Sequential([
        tf.keras.layers.Embedding(vocab,16,input_length=seq_len),
        tf.keras.layers.SimpleRNN(32),
        tf.keras.layers.Dense(vocab)
    ])
    opt=tf.keras.optimizers.Adam(learning_rate=0.03,clipnorm=clip if mode=="norm" else None,
                                 clipvalue=clip if mode=="value" else None)
    losses=[]; norms=[]
    for start in range(0,len(X),64):
        xb,yb=X[start:start+64],Y[start:start+64]
        with tf.GradientTape() as tape:
            logits=model(xb,training=True)
            loss=tf.reduce_mean(tf.keras.losses.sparse_categorical_crossentropy(
                yb,logits,from_logits=True))
        grads=tape.gradient(loss,model.trainable_variables)
        norms.append(float(tf.linalg.global_norm(grads)))
        opt.apply_gradients(zip(grads,model.trainable_variables))
        losses.append(float(loss))
    return losses,norms


results={}
for label,clip,mode in [("No clipping",None,None),("clipnorm 1.0",1.0,"norm"),
                        ("clipvalue 0.5",0.5,"value")]:
    results[label]=train(clip,mode)


for label,(loss,norm) in results.items():
    plt.plot(norm,label=label)
plt.xlabel("Training step"); plt.ylabel("Global gradient norm")
plt.title("Gradient Norm Comparison"); plt.legend(); plt.show()
for label,(loss,norm) in results.items():
    plt.plot(loss,label=label)
plt.xlabel("Training step"); plt.ylabel("Loss")
plt.title("Loss Comparison"); plt.legend(); plt.show()
for label,(loss,norm) in results.items():
```

```
    print(label,"maximum gradient norm:",round(max(norm),3),
          "NaN loss:",any(np.isnan(loss)))
```



Result

Result: Gradient norms and loss values were recorded for unclipped, clipnorm and clipvalue training. The setting with stable loss and controlled gradient norms is the recommended clipping option.

EXPERIMENT 18: RNN, LSTM AND GRU HYPERPARAMETER EXPERIMENTATION

Aim: To tune recurrent model settings for positive/negative sentiment classification using KerasTuner.

Algorithm

1. Load and pad IMDB movie reviews.
2. Create a model builder that selects SimpleRNN, LSTM or GRU.
3. Tune units, dropout, embedding size and learning rate.
4. Run a limited search for each cell type.
5. Compare best validation accuracy, parameter count and training time.

Python Code

```
# If needed, run once in Colab:
# !pip install -q keras-tuner


import tensorflow as tf
import keras_tuner as kt
import time, pandas as pd


vocab=10000; maxlen=150
(xtr,ytr),(xte,yte)=tf.keras.datasets.imdb.load_data(num_words=vocab)
xtr=tf.keras.preprocessing.sequence.pad_sequences(xtr,maxlen=maxlen)
xte=tf.keras.preprocessing.sequence.pad_sequences(xte,maxlen=maxlen)
```

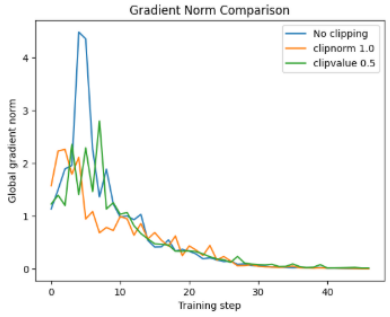
```
def make_builder(cell):
    def build(hp):
        units=hp.Choice("units",[32,64])
        emb=hp.Choice("embedding_size",[32,64])
        drop=hp.Choice("dropout",[0.2,0.4])
        lr=hp.Choice("learning_rate",[0.001,0.0005])
        m=tf.keras.Sequential([tf.keras.layers.Input((maxlen,)),
            tf.keras.layers.Embedding(vocab,emb)])
        layer={"RNN":tf.keras.layers.SimpleRNN,
               "LSTM":tf.keras.layers.LSTM,
               "GRU":tf.keras.layers.GRU}[cell]
        m.add(layer(units))
        m.add(tf.keras.layers.Dropout(drop))
        m.add(tf.keras.layers.Dense(1,activation="sigmoid"))
        m.compile(optimizer=tf.keras.optimizers.Adam(lr),
                  loss="binary_crossentropy",metrics=["accuracy"])
        return m
    return build

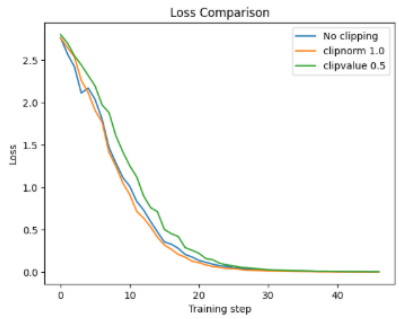

rows=[]
for cell in ["RNN","LSTM","GRU"]:
    tuner=kt.RandomSearch(make_builder(cell),objective="val_accuracy",
        max_trials=2,overwrite=True,directory="tuner_dir",project_name=cell)
    start=time.time()
    tuner.search(xtr[:12000],ytr[:12000],epochs=2,batch_size=128,
                 validation_split=.2,verbose=0)
    best=tuner.get_best_models(1)[0]
    loss,acc=best.evaluate(xte[:5000],yte[:5000],verbose=0)
    hp=tuner.get_best_hyperparameters(1)[0]
    rows.append([cell,round(acc,4),best.count_params(),round(time.time()-start,2),
                 hp.get("units"),hp.get("embedding_size"),
                 hp.get("dropout"),hp.get("learning_rate")])
print(pd.DataFrame(rows,columns=["Cell","Test accuracy","Parameters","Time (s)",
    "Units","Embedding","Dropout","Learning rate"]))
```

```
results={}
for label,clip,mode in [("No clipping",None,None),("clipnorm 1.0",1.0,"norm"),
                        ("clipvalue 0.5",0.5,"value")]:
    results[label]=train(clip,mode)

for label,(loss,norm) in results.items():
    plt.plot(norm,label=label)
plt.xlabel("Training step"); plt.ylabel("Global gradient norm")
plt.title("Gradient Norm Comparison"); plt.legend(); plt.show()
for label,(loss,norm) in results.items():
    plt.plot(loss,label=label)
plt.xlabel("Training step"); plt.ylabel("Loss")
plt.title("Loss Comparison"); plt.legend(); plt.show()
for label,(loss,norm) in results.items():
    print(label,"maximum gradient norm:",round(max(norm),3),
          "NaN loss:",any(np.isnan(loss)))
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
```






```
No clipping maximum gradient norm: 4.48 NaN loss: False
clipnorm 1.0 maximum gradient norm: 2.262 NaN loss: False
```

Result

Result: KerasTuner searched for suitable settings for SimpleRNN, LSTM and GRU. The best configurations were compared by test accuracy, parameter count and training time.

EXPERIMENT 19: PCA AND EXPLAINED VARIANCE ANALYSIS

Aim: To reduce image feature dimensions using PCA, verify the result with Scikit-learn and reconstruct images.

Algorithm

1. Load the handwritten digits dataset.
2. Standardise the pixel features.
3. Calculate PCA using covariance and eigen-decomposition.
4. Compare the explained variance with Scikit-learn PCA.
5. Plot scree/cumulative variance and reconstruct images using 10, 50 and 100 components.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
```

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler


digits=load_digits()
X=StandardScaler().fit_transform(digits.data)
cov=np.cov(X,rowvar=False)
values,vectors=np.linalg.eigh(cov)
order=np.argsort(values)[::-1]
values=values[order]; vectors=vectors[:,order]
ratio=values/values.sum()
cum=np.cumsum(ratio)
n95=np.searchsorted(cum,.95)+1
print("Components for 95% variance:",n95)


pca=PCA().fit(X)
print("Scikit-learn components for 95%:",
      np.searchsorted(np.cumsum(pca.explained_variance_ratio_),.95)+1)
plt.plot(np.arange(1,len(ratio)+1),ratio,marker=".")
plt.title("PCA Scree Plot"); plt.xlabel("Component"); plt.ylabel("Explained variance"); plt.show()
plt.plot(cum); plt.axhline(.95,color="red",linestyle="--")
plt.title("Cumulative Explained Variance"); plt.xlabel("Components"); plt.show()


fig,axes=plt.subplots(1,5,figsize=(10,2))
for i,ax in enumerate(axes):
    ax.imshow(vectors[:,i].reshape(8,8),cmap="gray"); ax.set_title(f"PC {i+1}"); ax.axis("off")
plt.show()


sample=X[:1]
fig,axes=plt.subplots(1,4,figsize=(9,2))
axes[0].imshow(sample.reshape(8,8),cmap="gray"); axes[0].set_title("Original")
for ax,k in zip(axes[1:],[10,50, min(100,X.shape[1])]):
    p=PCA(n_components=k)
    rec=p.inverse_transform(p.fit_transform(X[:]))
    ax.imshow(rec[0].reshape(8,8),cmap="gray"); ax.set_title(f"{k} PCs")
    ax.axis("off")
plt.show()
```

```
!pip install -q keras-tuner

import tensorflow as tf
import keras_tuner as kt
import time, pandas as pd

vocab=10000; maxlen=150
(xtr,ytr),(xte,yte)=tf.keras.datasets.imdb.load_data(num_words=vocab)
xtr=tf.keras.preprocessing.sequence.pad_sequences(xtr,maxlen=maxlen)
xte=tf.keras.preprocessing.sequence.pad_sequences(xte,maxlen=maxlen)

def make_builder(cell):
    def build(hp):
        units=hp.Choice("units",[32,64])
        emb=hp.Choice("embedding_size",[32,64])
        drop=hp.Choice("dropout",[0.2,0.4])
        lr=hp.Choice("learning_rate",[0.001,0.0005])
        m=tf.keras.Sequential([tf.keras.layers.Input((maxlen,)),
            tf.keras.layers.Embedding(vocab,emb)])
        layer={"RNN":tf.keras.layers.SimpleRNN,
               "LSTM":tf.keras.layers.LSTM,
               "GRU":tf.keras.layers.GRU}[cell]
        m.add(layer(units))
        m.add(tf.keras.layers.Dropout(drop))
        m.add(tf.keras.layers.Dense(1,activation="sigmoid"))
        m.compile(optimizer=tf.keras.optimizers.Adam(lr),
                  loss="binary_crossentropy",metrics=["accuracy"])
        return m
    return build

rows=[]
for cell in ["RNN","LSTM","GRU"]:
    tuner=kt.RandomSearch(make_builder(cell),objective="val_accuracy",
        max_trials=2,overwrite=True,directory="tuner_dir",project_name=cell)
    start=time.time()
    tuner.search(xtr[:12000],ytr[:12000],epochs=2,batch_size=128,
                 validation_split=.2,verbose=0)
    best=tuner.get_best_models(1)[0]
    loss,acc=best.evaluate(xte[:5000],yte[:5000],verbose=0)
    hp=tuner.get_best_hyperparameters(1)[0]
    rows.append([cell,round(acc,4),best.count_params(),round(time.time()-start,2),
                 hp.get("units"),hp.get("embedding_size"),
                 hp.get("dropout"),hp.get("learning_rate")])
print(pd.DataFrame(rows,columns=["Cell","Test accuracy","Parameters","Time (s)",
    "Units","Embedding","Dropout","Learning rate"]))
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.13/dist-packages/keras/src/saving/saving_lib.py:797: UserWarning: Skipping variable loading for optimizer 'adam', because it has 2 variables whereas the saved optimizer has 14 variables.
  saveable.load_own_variables(weights_store.get(inner_path))
   Cell  Test accuracy  Parameters  Time (s)  Units  Embedding  Dropout  \
0   RNN        0.7848      643137     26.24     32         64      0.2
1  LSTM        0.8422      344897     57.28     64         32      0.4
2   GRU        0.8254      665025     71.12     64         64      0.2

   Learning rate
0          0.001
1          0.001
2          0.001
```

Result

Result: PCA reduced the image features. The scree plot and cumulative variance identified the components needed for 95% variance, and reconstructed images showed the effect of using different component counts.

EXPERIMENT 20: PROBABILISTIC PCA AND RECONSTRUCTION ANALYSIS

Aim: To apply Probabilistic PCA to noisy sensor data, estimate noise variance, reconstruct readings and generate new samples.

Algorithm

1. Generate multivariate sensor readings and add Gaussian noise.
2. Use the PPCA model $x = Wz + \mu + \varepsilon$, where ε has variance σ^2.
3. Fit PCA and obtain its estimated noise variance.
4. Reconstruct data using different component counts and calculate error.
5. Generate synthetic sensor samples from the fitted Gaussian model.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error


np.random.seed(8)
n=500
latent=np.random.randn(n,2)
W=np.array([[2,0.5],[1,1.5],[0.5,2],[1.2,0.3],[0.4,1.0]])
mu=np.array([25,4,100,12,60])
clean=latent@W.T+mu
sensor=clean+np.random.normal(0,0.8,clean.shape)


sc=StandardScaler()
X=sc.fit_transform(sensor)
# PPCA is fitted using PCA; noise_variance_ estimates residual noise.
pca=PCA(n_components=2).fit(X)
print("Estimated noise variance:",pca.noise_variance_)


errors=[]
for k in [1,2,3,4]:
    model=PCA(n_components=k)
    scores=model.fit_transform(X)
    reconstructed=model.inverse_transform(scores)
    err=mean_squared_error(X,reconstructed)
    errors.append(err)
    print("Components:",k,"Reconstruction MSE:",round(err,4))
plt.plot([1,2,3,4],errors,marker="o")
plt.xlabel("Number of components"); plt.ylabel("Reconstruction MSE")
plt.title("PPCA/PCA Reconstruction Error"); plt.show()


# Generate new samples from the fitted PCA Gaussian approximation.
new_scores=np.random.randn(5,2)*np.sqrt(pca.explained_variance_)
new_scaled=pca.mean_+new_scores@pca.components_
new_sensor=sc.inverse_transform(new_scaled)
print("Generated synthetic sensor readings:\n",np.round(new_sensor,2))
```

```
print("Sensor columns: temperature, vibration, pressure, current, humidity")
```

```
from sklearn.metrics import mean_squared_error

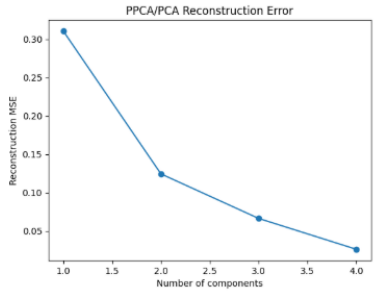
np.random.seed(8)
n=500
latent=np.random.randn(n,2)
W=np.array([[2,0.5],[1,1.5],[0.5,2],[1.2,0.3],[0.4,1.0]])
mu=np.array([25,4,100,12,60])
clean=latent@W.T+mu
sensor=clean+np.random.normal(0,0.8,clean.shape)

sc=StandardScaler()
X=sc.fit_transform(sensor)
# PPCA is fitted using PCA; noise_variance_ estimates residual noise.
pca=PCA(n_components=2).fit(X)
print("Estimated noise variance:",pca.noise_variance_)

errors=[]
for k in [1,2,3,4]:
    model=PCA(n_components=k)
    scores=model.fit_transform(X)
    reconstructed=model.inverse_transform(scores)
    err=mean_squared_error(X,reconstructed)
    errors.append(err)
    print("Components:",k,"Reconstruction MSE:",round(err,4))
plt.plot([1,2,3,4],errors,marker="o")
plt.xlabel("Number of components"); plt.ylabel("Reconstruction MSE")
plt.title("PPCA/PCA Reconstruction Error"); plt.show()

# Generate new samples from the fitted PCA Gaussian approximation.
new_scores=np.random.randn(5,2)*np.sqrt(pca.explained_variance_)
new_scaled=pca.mean_+new_scores@pca.components_
new_sensor=sc.inverse_transform(new_scaled)
print("Generated synthetic sensor readings:\n",np.round(new_sensor,2))
print("Sensor columns: temperature, vibration, pressure, current, humidity")
```

```
Estimated noise variance: 0.2076032839062167
Components: 1 Reconstruction MSE: 0.3107
Components: 2 Reconstruction MSE: 0.1243
Components: 3 Reconstruction MSE: 0.0663
Components: 4 Reconstruction MSE: 0.026
```




```
Generated synthetic sensor readings:
 [[ 23.14   2.09  98.14  10.85  58.84]
 [ 24.5    4.92 101.57  11.5   60.87]
 [ 25.88   2.82  97.82  12.84  58.78]
 [ 22.18   2.45  99.13  10.07  59.38]
 [ 23.27   2.82  99.13  10.85  59.42]]
Sensor columns: temperature, vibration, pressure, current, humidity
```

Result

Result: The PPCA noise variance was estimated, reconstruction error was measured for different component counts, and new synthetic sensor readings were generated from the fitted model.